

A Practical Introduction to Retrieval Augmented Generation

A short guide to the PDF retrieval demo in this project

Retrieval augmented generation (RAG) combines document search with a language model. Instead of asking a model to rely only on information stored in its parameters, an application first retrieves passages from a selected knowledge source. Those passages can then be shown to the user or supplied to a language model as grounded context.

The RAG workflow

A typical RAG system has an indexing stage and a question stage. Indexing prepares the knowledge base before questions arrive; retrieval runs for each question.

- Load a source document and extract its text. For PDFs, extraction works when the page contains selectable text; scanned pages generally need OCR first.
- Split long text into smaller chunks. Chunks should preserve enough context to be understandable, while remaining focused enough to retrieve accurately.
- Encode each chunk as a numeric embedding, a vector whose nearby values represent related meanings.
- Store the vectors alongside their original text in a vector index.
- Embed a user question, search for the nearest chunk vectors, and return the matching source passages.

Chunking and overlap

A chunk that is too large may mix unrelated topics, while a chunk that is too small may omit important context. This project uses character-based chunks with a default size of 1,000 characters and an overlap of 150 characters. The overlap repeats a small amount of text at chunk boundaries, helping keep a sentence or idea together when a document is split. These values are a starting point and should be tuned for the source material.

Embeddings and similarity search

The project uses the sentence-transformers/all-MiniLM-L6-v2 model to create embeddings. It normalizes vectors and stores them in a FAISS IndexFlatIP index. For normalized vectors, inner product ranks results by cosine similarity. The search returns up to the requested number of matching text chunks; it does not calculate a natural-language answer by itself.

Using the project API

Run `uvicorn app.api:app --reload` from the project directory, then open <http://127.0.0.1:8000/docs>. Upload a PDF to **POST /documents** using the form field **file**. Ask a question at **POST /ask** with JSON such as `{"question": "How does RAG find information?", "top_k": 3}`. The response contains ranked passages retrieved from the indexed PDF. If no upload has been made, the API indexes this sample PDF the first time a question is asked.

Practical limitations

- Retrieval quality depends on text extraction, chunk boundaries, and how well the embedding model represents the question and document.
- The current index is held in memory. It is lost when the API process stops, so uploaded documents must be indexed again after a restart.
- The API returns evidence passages rather than a generated answer. A production answer-generation layer should cite the passages and avoid claims unsupported by them.
- Documents may contain sensitive data. Only upload files that are appropriate for the environment where the service is running.

Key idea

RAG makes a model or search application more useful by connecting a question to relevant source material. Reliable results depend on preparing readable documents, retrieving the right chunks, and clearly separating retrieved evidence from generated text.